Business Requirements Document

Peelen MS Backend - Authentication, Operations, and Reporting

Document version: 1.0 **Prepared:** 13 October 2026 **Source of truth:** current application routes, handlers, use cases, `ddl/init.sql`, seed files, and Postman collection in this repository. **Audience:** product owners, operations, engineering, QA, and database administrators.

This is an implementation-derived BRD, not a replacement for an approved product PRD. It documents behavior that is present in the current code and schema. Where the implementation or business policy is incomplete, the gap is called out instead of being represented as a delivered capability.

1. Executive summary

Peelen MS BE is a Go/Gin REST backend with PostgreSQL persistence. It provides authentication, session management, role and scope administration, operational master data, assignment-based worker tasks, pipeline and inventory reports, and daily reporting summaries. Optional MinIO object storage handles inventory images and policy PDFs.

The API is versioned under `/api/v1`. Authenticated records are restricted according to roles and assigned site/vendor scopes. Database deletes for sites, vendors, and operational master data are implemented as status changes (`INACTIVE`), not physical deletion.

2. Business objectives

1. Allow authorized administrators to provision users and control access by role and site/vendor scope.
2. Maintain shared site/vendor, inventory, policy, item, and pipeline master data.
3. Give workers access to their active task assignments and allow them to submit task and inventory reports.
4. Give managers visibility into scoped reports and support review/status updates.
5. Provide date-based dashboard and daily pipeline summaries.
6. Keep an audit trail for authentication-sensitive and operational report state changes.

3. Scope

3.1 In scope

- Username/password login, JWT access token, rotating refresh token, logout, password change, and session revocation.
- Four roles: `SUPER_ADMIN`, `ADMIN`, `ADMIN_WORKER`, and `WORKER`.
- User administration, role assignment, and site/vendor scopes.
- Site/vendor management with vendor-to-site many-to-many association.
- Inventory, policy, shared item, pipeline, and pipeline-item APIs.
- Active-assignment worker task list.
- Pipeline report submission, detail updates, review statuses, and history.
- Inventory request/damage report creation and manager review.
- Scope-filtered dashboard and daily reports.
- MinIO inventory image and policy PDF upload plus short-lived signed download links.

3.2 Out of scope or not yet delivered

- UI/mobile client, notifications, offline synchronization, and external identity provider.
- Upload endpoints for pipeline/inventory-report evidence. Report fields currently accept object keys; evidence file upload is not implemented.
- Assignment management endpoints. Assignment tables exist and seed data creates assignments, but no API route currently creates/updates assignments.
- Pipeline-report history read endpoint.
- Inventory-report audit/history table or history endpoint.
- CSV/XLSX/PDF report export.
- SLA, availability, throughput, retention, and disaster recovery objectives; not specified by code/schema.
- Multi-tenant partitioning beyond site/vendor scoping.

4. Stakeholders and actors

Actor	Responsibility	Access model
Super Admin	Global administration, master data, user roles/scopes, report oversight	Global; no scope row required
Admin	Site operations and review	Limited to assigned sites and vendors mapped to those sites
Admin Worker	Vendor operations, worker administration, report review	Limited to assigned vendor(s); can manage pipeline items but not pipelines
Worker	Execute assigned work and submit reports	Own active task assignments and own reports; inventory reports limited to own vendor scope
Service operator / DBA	Configure environment, database, TLS, MinIO, backups	Infrastructure/database access outside business API

5. Business workflows

5.1 Authentication and onboarding

1. An authorized administrator creates a user and assigns role/scope values.
2. The user logs in with username/password.
3. The service returns an access token and sets a Secure, HttpOnly, SameSite=Strict refresh cookie.
4. A user marked `must_change_password` may access password change, profile, and logout flows, but is blocked from other authenticated endpoints until changing the password.
5. Login failures are rate-limited and account lockout is enforced. Refresh token rotation detects reuse and can revoke sessions.

5.2 Master data and scope

1. Super Admin creates sites and vendors and relates vendors to one or more sites.
2. Super Admin assigns site scopes to Admins and vendor scopes to Admin Workers/Workers.
3. Admins see vendors connected to assigned sites. Vendor-scoped roles see their vendors and corresponding sites.
4. Super Admin manages shared items. Super Admin/Admin manage pipelines according to global/site scope. Admin Worker manages pipeline-items in vendor scope.
5. Deactivation sets a resource status to `INACTIVE`; records remain stored.

5.3 Worker task and pipeline reporting

1. An assignment is created in `pipeline_item_assignments` by a privileged data/process. An assignment is active only while `is_active = true`.
2. Worker requests `/me/task` for a date and sees active assigned pipeline items plus any report status for that date.
3. Worker submits a report for the assigned pipeline item, shift, and report date.
4. Report details may include evidence object keys, open/closed timestamps, category, and notes.

5. A manager in scope can update report status/revision state. Status/revision transitions are appended to `pipeline_report_history`.

5.4 Inventory reporting

1. A scoped user submits `REQUEST` or `DAMAGED` report for a vendor. `DAMAGED` must reference an inventory record owned by that vendor.
2. Admin, Admin Worker, or Super Admin reviews the report, sets a review status, and may add a reason.
3. Users read reports according to scope; ordinary Worker reads are restricted to own reports.

5.5 Dashboard and daily reporting

1. An authenticated user requests counts for a date (defaults to current UTC date).
2. Optional site/vendor filters further narrow, but do not broaden, the caller's access.
3. Dashboard returns pipeline-report and inventory-report status counts.
4. Daily report returns task/report/completion counts grouped by vendor and pipeline.

6. Functional requirements and acceptance statements

ID	Requirement	Acceptance statement
BR-001	Authenticate users securely	Valid credentials establish a session and return an access token; invalid credentials do not reveal account existence.
BR-002	Manage sessions	Refresh tokens rotate; logout/revocation invalidates applicable sessions.
BR-003	Enforce authorization scope	Every protected read/write uses authenticated roles and site/vendor scope; out-of-scope records are not exposed.
BR-004	Administer users	Super Admin manages accounts, roles, and scopes; Admin Worker can manage permitted worker accounts only within vendor scope.
BR-005	Maintain sites/vendors	Super Admin creates/updates/deactivates; authorized users can read only their related site/vendor scope.
BR-006	Maintain operational inventory and policies	Authorized roles manage scoped inventory/policy data; uploaded files are validated and stored as object keys.
BR-007	Maintain pipelines	Super Admin manages globally and Admin manages in site scope; Admin Worker does not manage pipelines.
BR-008	Maintain shared inspection items	Only Super Admin mutates shared item master data.
BR-009	Provide task list	User sees only active assignments for themselves and requested report date.
BR-010	Submit pipeline reports	Report is accepted only for an active assignment matching user, pipeline item, and shift; duplicate user/task/shift/date is rejected by database uniqueness.
BR-011	Review pipeline reports	Scoped manager can update report status/revision; every status/revision change has a history row.
BR-012	Submit/review inventory reports	Report vendor/inventory consistency is enforced by scope and composite FK; only manager roles review.
BR-013	Provide reporting summaries	Dashboard and daily report apply scope and date filters and return aggregated counts.
BR-014	Audit important changes	Auth/account changes and pipeline-report state changes are persisted with actor/time; API error responses are logged.
BR-015	Keep inactive data	API DELETE operations on supported master resources deactivate rather than physically remove records.

7. Roles and access matrix

Legend: **G** = global; **S** = own site/vendor scope; **A** = active assignment/own report; **R** = read; **W** = write; **Review** = review/update statuses; **—** = unavailable.

Capability	Super Admin	Admin	Admin Worker	Worker
User list/create/update/reset password	G	—	S (worker accounts)	—
Set roles/scopes/revoke any user's sessions	G	—	—	—
Read sites/vendors	G	S	S	S
Create/update/deactivate sites/vendors	W	—	—	—
Inventory/policy read	G	S	S	S
Inventory/policy create/update/deactivate	G	S	S	—
Shared item read	G	Assigned/in-scope	Assigned/in-scope	A
Shared item create/update/deactivate	W	—	—	—
Pipeline read	G	S	S through scope/assignment behavior	A
Pipeline create/update/deactivate	G	S	—	—
Pipeline-item read	G	S	S	A
Pipeline-item create/update/deactivate	G	S	S	—
My active task list	Own user	Own user	Own user	Own user
Pipeline report read	G	S	S	A / own
Pipeline report submit/detail update	As authorized by role and active assignment for worker submit	Scoped	Scoped	A
Pipeline report review	G	S	S	—
Inventory report read	G	S	S	Own reports
Inventory report create	G	S	S	Own vendor
Inventory report review	G	S	S	—
Dashboard/daily summaries	G	S	S	Own reporting scope

Implementation note: Middleware authenticates routes; detailed authorization also occurs in use cases/repositories. Role combinations can affect effective capability. Verify multi-role users against the current implementation when provisioning.

8. API catalog

All routes are prefixed by `/api/v1`. Except health, login, and refresh, routes require `Authorization: Bearer <access_token>`. JSON endpoints use `application/json`; inventory/policy file endpoints use `multipart/form-data`.

8.1 Health and authentication

Method	Endpoint	Access	Purpose / request
GET	`/health`	Public	Liveness response `{ "status": "ok" }`
POST	`/auth/login`	Public	`{ "username": "...", "password": "..." }`; returns access token/user and refresh cookie
POST	`/auth/refresh`	Public	Rotates refresh cookie; optional body `{ "refresh_token": "..." }` for non-browser client
POST	`/auth/logout`	Authenticated	Revoke current access/refresh session
POST	`/auth/logout-all`	Authenticated	Revoke all sessions for current user
GET	`/auth/me`	Authenticated	Current profile, roles, and scopes
PUT	`/auth/password`	Authenticated	`{ "current_password": "...", "new_password": "..." }`

8.2 User and administrator APIs

Method	Endpoint	Access	Purpose / request
GET	`/users`	Super Admin, Admin Worker	List users allowed to actor
POST	`/users`	Super Admin, Admin Worker	Create user; username, roles required; optional profile/password/site/vendor scopes
GET	`/users/{id}`	Super Admin, Admin Worker	Read managed user
PATCH	`/users/{id}`	Super Admin, Admin Worker	Update profile/active state
POST	`/users/{id}/reset-password`	Super Admin, Admin Worker	`{}` generates temporary password; optional explicit password
PUT	`/users/{id}/roles`	Super Admin	Update roles; optional scopes
PUT	`/users/{id}/scopes`	Super Admin	Update scopes; optional roles
DELETE	`/users/{id}/sessions`	Super Admin	Revoke user's sessions
GET	`/roles`	Super Admin	List role catalog
GET	`/audit-logs`	Super Admin	Filter audit history by user/action/entity/date and limit

8.3 Sites, vendors, and tasks

Method	Endpoint	Access	Purpose / request
GET	`/sites`	Authenticated, scope-filtered	List related sites
POST	`/sites`	Super Admin	Create site: `code`, `name`, optional `site_type`, `status`
GET	`/sites/{id}`	Authenticated, scope-filtered	Get site
PATCH	`/sites/{id}`	Super Admin	Update site fields
DELETE	`/sites/{id}`	Super Admin	Soft-deactivate site
GET	`/vendors`	Authenticated, scope-filtered	List related vendors and associated site IDs
POST	`/vendors`	Super Admin	Create vendor: `code`, `name`, optional `site_ids`, `status`
GET	`/vendors/{id}`	Authenticated, scope-filtered	Get vendor
PATCH	`/vendors/{id}`	Super Admin	Update vendor and optionally replace `site_ids`
DELETE	`/vendors/{id}`	Super Admin	Soft-deactivate vendor
GET	`/me/task?date=YYYY-MM-DD`	Authenticated	Own active assignment list and report state for date; date defaults to UTC today
GET	`/me/tasks?date=YYYY-MM-DD`	Authenticated alias	Same behavior as `/me/task`

8.4 Inventory, policies, items, pipelines

Method	Endpoint	Access	Purpose / request
GET	`/inventories`	Authenticated, scope-filtered	Paginated inventory; supports `page`, `limit`, `status`, `q`, `vendor_id`, `site_id`, `status_condition`
POST	`/inventories`	Super Admin, Admin, Admin Worker	Multipart fields `vendor_id`, `item`, `jenis_item`, `qty`, `status_condition`; optional JPEG/PNG/WebP `image` (5 MiB max)
GET	`/inventories/{id}`	Authenticated, scope-filtered	Inventory details; signed `image_url` for active record
PATCH	`/inventories/{id}`	Super Admin, Admin, Admin Worker	Multipart partial update; optional image replacement
DELETE	`/inventories/{id}`	Super Admin, Admin, Admin Worker	Soft-deactivate
GET	`/policies`	Authenticated, scope-filtered	Paginated policy list
POST	`/policies`	Super Admin, Admin, Admin Worker	Multipart `vendor_id`, `name`, required PDF `file` (10 MiB max)
GET	`/policies/{id}`	Authenticated, scope-filtered	Policy details
GET	`/policies/{id}/file`	Authenticated, scope-filtered	Signed download URL, 10-minute expiry; active policies only
PATCH	`/policies/{id}`	Super Admin, Admin, Admin Worker	Multipart partial update; optional PDF replacement
DELETE	`/policies/{id}`	Super Admin, Admin, Admin Worker	Soft-deactivate
GET	`/items`	Authenticated, in-scope/assignment-filtered	List shared item master
POST	`/items`	Super Admin	JSON `name`, optional `description`
GET	`/items/{id}`	Authenticated, in-scope/assignment-filtered	Get item
PATCH	`/items/{id}`	Super Admin	Update name/description/status
DELETE	`/items/{id}`	Super Admin	Soft-deactivate
GET	`/pipelines`	Authenticated, scope-filtered	List pipelines
POST	`/pipelines`	Super Admin, Admin in scope	JSON `vendor_id`, `name`, optional `description`
GET	`/pipelines/{id}`	Authenticated, scope-filtered	Get pipeline
PATCH	`/pipelines/{id}`	Super Admin, Admin in scope	Update pipeline
DELETE	`/pipelines/{id}`	Super Admin, Admin in scope	Soft-deactivate
GET	`/pipelines/{id}/items`	Authenticated, scope/assignment-filtered	List tasks in pipeline
POST	`/pipelines/{id}/items`	Super Admin, Admin, Admin Worker in scope	JSON `item_id`, optional `sort_order`
GET	`/pipeline-items/{id}`	Authenticated, scope/assignment-filtered	Get pipeline item
PATCH	`/pipeline-items/{id}`	Super Admin, Admin, Admin Worker in scope	Update `sort_order`/`status`
DELETE	`/pipeline-items/{id}`	Super Admin, Admin, Admin Worker in scope	Soft-deactivate pipeline item

8.5 Reports and summaries

Method	Endpoint	Access	Purpose / request
GET	`/pipeline-reports`	Authenticated, scoped	Paginated reports; supports page, `limit`, `vendor_id`, `site_id`, `user_id`, `status`, `from`, `to`
POST	`/pipeline-reports`	Authenticated; worker must have active matching assignment	JSON `pipeline_item_id`, `shift`, `report_date`; optional status and report details
GET	`/pipeline-reports/{id}`	Authenticated, scoped	Read report and detail fields
PATCH	`/pipeline-reports/{id}`	Assigned worker or in-scope manager	Update report details/status/revision; status changes create history
GET	`/inventory-report`	Authenticated, scoped	Paginated inventory reports
POST	`/inventory-report`	Authenticated in vendor scope	JSON `vendor_id`, `report_type`, `description`; optional `inventory_id`, evidence object key
GET	`/inventory-report/{id}`	Authenticated, scoped	Read inventory report
PATCH	`/inventory-report/{id}/review`	Super Admin, Admin, Admin Worker in scope	JSON status `IN_REVIEW`, `APPROVED`, `REJECTED`, or `RESOLVED`; optional `review_reason`
GET/POST	`/inventory-reports`	Alias	Same list/create handlers as singular path
GET	`/inventory-reports/{id}`	Alias	Same detail handler
PATCH	`/inventory-reports/{id}/review`	Alias	Same review handler
GET	`/dashboard/report?date=...&site_id=...&vendor_id=...`	Authenticated, scope-filtered	Pipeline and inventory report counts for date; date defaults to UTC today
GET	`/reports/daily?date=...&site_id=...&vendor_id=...`	Authenticated, scope-filtered	Counts grouped by vendor and pipeline

Report statuses

- Pipeline: `PENDING`, `IN_PROGRESS`, `SUBMITTED`, `COMPLETED`, `REJECTED`.
- Pipeline revision: `NONE`, `REQUESTED`, `RESUBMITTED`.
- Inventory report type: `REQUEST`, `DAMAGED`.
- Inventory report: `SUBMITTED`, `IN_REVIEW`, `APPROVED`, `REJECTED`, `RESOLVED`.
- Safety note category: `NEAR_MISS`, `UNSAFE_ACTION`, `UNSAFE_CONDITION`.

Report list responses use `{ "data": [], "page": 1, "limit": 20, "total": 0 }`. Errors use `{ "error": { "code": "...", "message": "..." } }`.

9. Data model and table relationships

9.1 Relationship overview

```
users --< user_roles >-- roles
users --< user_scopes >-- sites
users --< user_scopes >-- vendors
vendors --< vendor_sites >-- sites
vendors --< inventories
vendors --< policies
vendors --< pipelines --< pipeline_items >-- items
pipeline_items --< pipeline_item_assignments >-- users
pipeline_item_assignments --< pipeline_reports -- 0..1 pipeline_report_details
pipeline_reports --< pipeline_report_history
vendors --< inventory_reports >-- users
inventory_reports -- optional inventory reference (composite inventory_id/vendor_id)
```

`users` to `roles` and `users` to site/vendor scopes are many-to-many relations. A vendor may serve multiple sites and a site may be served by multiple vendors. Pipeline reports are unique per pipeline item, user, shift, and report date. The assignment composite foreign key ensures the report corresponds to an assignment.

9.2 Table catalog

Table	Primary key	Important columns / constraints	Relationships / purpose
users	id UUID	username unique/lowercase; password_hash; profile; active/lock/first-password-change flags; timestamps	User identity referenced by auth, scope, report, audit, and assignment tables
roles	id smallint	code unique; allowed four role codes	Role lookup
user_roles	(user_id, role_id)	assigned_at, optional assigned_by	User-role many-to-many
sites	id UUID	code unique, name, optional site_type, status, creator/timestamps	Operational locations
vendors	id UUID	code unique, name, status, creator/timestamps	Contractor/vendor master
user_scopes	id UUID	nullable site_id or vendor_id, at least one required; unique partial indexes prevent duplicate single-kind scope	Connects users to authorized sites/vendors
vendor_sites	(vendor_id, site_id)	optional creator and created time	Vendor/site many-to-many
sessions	id UUID	unique token_hash, expiry/revocation/activity, user agent, IP	Login session tracking
refresh_tokens	id UUID	user_id, session_id, unique token hash, expiry/revocation	Rotating refresh token store; cascades with session/user
audit_logs	id bigserial	action, entity/id, before/after JSONB, actor, IP, time	Authentication and administrative audit events
inventories	id UUID	vendor, item/type, nonnegative quantity, object key img, condition enum, status; unique (id,vendor_id)	Vendor-owned inventory
policies	id UUID	vendor, name, required PDF object key, status	Vendor policy/SOP master
items	id UUID	shared name/description/status	Reusable task definition
pipelines	id UUID	vendor, name/description/status	Vendor workflow/checklist
pipeline_items	id UUID	pipeline, item, nonnegative sort order/status; unique pipeline/item and (id,pipeline_id)	Ordered pipeline task items
pipeline_item_assignments	id UUID	pipeline item/user/shift, active flag, assigner; unique (pipeline_item_id,user_id,shift)	Worker task assignment
pipeline_reports	id UUID	task/user/shift/date, status/revision enums; unique (pipeline_item_id, user_id,shift,report_date)	Per-assignment daily task outcome
pipeline_report_details	id UUID	unique report ID, evidence object keys, open/close timestamps, note category/text; close must not precede open	Optional report detail record
pipeline_report_history	id bigserial	actor, previous/new status and revision, reason, time	Append-only status/revision transition history
inventory_reports	id UUID	vendor/inventory/user, type, evidence key, description, review state/reason/reviewer	Inventory request/damage report; DAMAGED requires inventory

9.3 Key integrity rules

- UUIDs are used for business records; audit and transition history use `bigserial`.
- `ON DELETE CASCADE` is reserved for user/role/scope/session associations where appropriate. Operational history/report foreign keys use restrictive deletion.
- Inventory reports reference inventory by `(inventory_id, vendor_id)` to prevent cross-vendor mismatch.
- Pipeline report FK `(pipeline_item_id,user_id,shift)` must match a declared assignment.
- Soft deactivation preserves references and history.
- Object storage columns hold object keys, not file bytes or public URLs.

10. Security, privacy, and operational requirements

- Passwords are stored as bcrypt hashes; refresh/access secrets are not stored as plaintext token values in token tables.

- API authentication uses bearer JWT access tokens. Refresh cookie is Secure, HttpOnly, SameSite=Strict, scoped to `/api/v1/auth`.
- HTTPS is enforced in release mode, either by direct TLS or trusted TLS-terminating proxy configuration.
- Login/refresh attempts are rate-limited in process memory; distributed deployments need shared rate limiting for a global quota.
- Site/vendor scope predicates must be applied in repositories, not only in client/UI filtering.
- PDF upload maximum is 10 MiB and PDF signature is checked. Inventory images allow JPEG/PNG/WebP up to 5 MiB.
- Signed download URLs expire after 10 minutes.
- Demo seed accounts/password are strictly for development/testing; never load demo seeds into production.
- The application currently has no documented data retention, backup, RPO/RTO, or SLA policy; owners must define these before production readiness.

11. Assumptions, decisions, and open gaps

1. Role names and current scope rules reflect the implementation, and require product-owner approval as the business policy.
2. A worker's task report date uses UTC date behavior where no date is supplied.
3. Inventory report evidence and pipeline report evidence are object keys; clients need an independent upload path until evidence upload APIs are delivered.
4. Assignment creation/reassignment is currently performed outside these API routes.
5. Dashboard counts are operational summaries, not an accounting/official compliance report.
6. Status values and state transitions should be confirmed with product owners; database enum checks constrain allowed values but do not encode a complete transition state machine.
7. Endpoint integration tests currently verify route registration and unit tests; full API-to-PostgreSQL runtime verification depends on a provisioned integration database.
8. Schema initialization uses idempotent `CREATE TABLE IF NOT EXISTS` for many objects, but does not automatically reconcile every pre-existing schema version. Production upgrades should use reviewed migrations and backup/restore procedures.

12. Acceptance and verification checklist

☐ Product owner signs off roles, scope semantics, report status transitions, and date/timezone convention.

☐ QA verifies every documented route, including forbidden/out-of-scope cases.

☐ PostgreSQL integration tests cover duplicate reports, assignment matching, FK constraints, and report aggregates.

☐ MinIO integration tests cover MIME/signature validation, upload failures, orphan cleanup, and signed URL expiry.

☐ Security review verifies JWT, refresh reuse, HTTPS proxy trust, input validation, and scope isolation.

☐ Operations define secret rotation, logging/PII policy, monitoring, backups, retention, and recovery objectives.

☐ Production database migration is reviewed independently; demo seed is excluded.

Appendix A. Related repository artifacts

- Schema: `ddl/init.sql`
- Authentication/development seed: `ddl/seed.sql`
- Operational development fixtures: `ddl/seed_v2.sql`
- API test collection: `ddl/peelen-ms-be.postman_collection.json`
- API and setup notes: `readme.md`